Gym-OS

نظام إدارة الجيم — دليل الفريق لفهم المشروع بالكامل

Gym Management System – Console Application in Java

Java 21 | SQLite (JDBC) | 55 Java Files | 13 DB Tables | 19 Business Rules | 3 Login Roles

شرح مبسط لكل ملف · قواعد العمل 1–19 · الإصلاحات التي تمت · أسئلة متوقعة وأجوبتها

AGENDA

محتويات الدليل

1. **نظرة عامة على المشروع والبنية (Architecture)**
   Overview & Architecture
2. **الملفات الأساسية: الدخول، القوائم، قواعد العمل**
   Core files: entry, menus, rules
3. **قواعد العمل التسعة عشر (Rules 1–19)**
   Business rules
4. **طبقة البيانات: SQLite + DAO + الموديلات**
   DAO layer, DB, models
5. **ماذا أصلحنا ولماذا (ملخص الإصلاحات)**
   What we fixed & why
6. **أسئلة متوقعة وأجوبة قصيرة + سيناريو العرض**
   Q&A prep + demo script

01

نظرة عامة والبنية الأساسية

OVERVIEW & ARCHITECTURE

PROJECT OVERVIEW

ما هو المشروع؟

تطبيق لإدارة جيم كامل يعمل من الطرفية (Terminal) — بدون واجهات رسومية. كل شيء عن طريق قوائم مرقمة يتفاعل معها المستخدم، وكل البيانات تُخزن في قاعدة بيانات SQLite واحدة (gym.db).

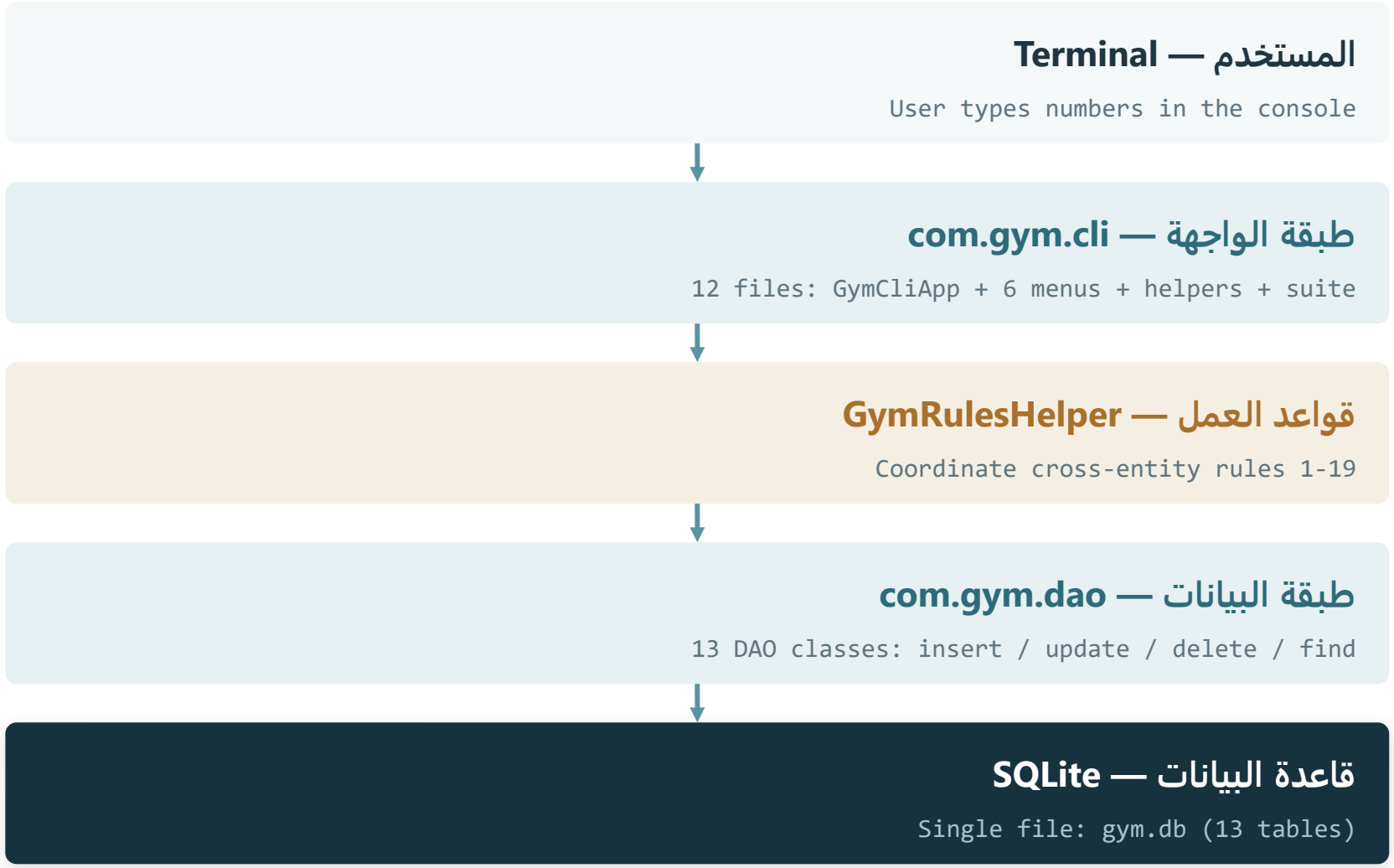
55	13	19	3	22
ملف Java	جدول في قاعدة البيانات	قاعدة عمل	أدوار دخول (Roles)	سيناريو تحقق آلي

الوظائف الرئيسية:

- إدارة الأعضاء وملفاتهم الطبية والتقدم (Members / Medical / Progress)
- المدربون ومواعيدهم المتاحة والاشتراكات (Trainers / Slots / Subscriptions)
- العضويات والمدفوعات والاستبدال (Memberships / Payments / Void)
- الحجز وجدول التدريب وبرامج التمرين (Sessions / Workout HUD)
- الحضور والمعدات وخريطة الصالة (Attendance / Equipment / Floor Map)
- حسابات الموظفين والصلاحيات والنسخ الاحتياطي (Staff RBAC / Backup)

ARCHITECTURE

البنية: أربع طبقات واضحة



قاعدة التصميم

- القوائم لا تلمس SQL مباشرة — كل قراءة/كتابة تمر عبر DAO.
- قواعد العمل المعقدة (تحتاج أكثر من جدول) في GymRulesHelper.
- كل DAO يتحقق من صحة الحقول في validate() قبل الكتابة.
- الأخطاء ترجع كـ ValidationException أو DatabaseException وتُطبع برسالة ودودة.

PACKAGE STRUCTURE

حزم المشروع الخمس (Packages)

com.gym.cli ملف	12	كل ما يراه المستخدم: الدخول، القوائم الست، أدوات الإدخال، بذور البيانات، حزمة التحقق.
com.gym.dao ملف	13	Data Access Objects: صنف لكل جدول — insert / update / delete / findById / findAll + validate.
com.gym.model ملف	25	14 كيان (Customer, Trainer, Payment...) + 11 نوع تعدادي (enum) للحالات.
com.gym.exception ملفات	3	GymException الأب + DatabaseException + ValidationException.
com.gym.util ملفان	2	DatabaseConnection (إنشاء الجداول والاتصال) + SerializationUtil (نسخ ser.).

run.bat — BUILD & RUN

كيف يُبنى ويُشغَّل المشروع؟

1

الانتقال إلى مجلد المشروع

يُقرأ من مكان صحيح دائماً gym.db يضمن أن — cd /d "%~dp0"

2

جمع كل ملفات المصدر

(ملفاً 55) dir /s /b src*.java > out\sources.txt

3

الترجمة (Compile)

javac --release 21 -encoding UTF-8 -Xlint:all -cp "lib*" -d out @out\sources.txt

4

التشغيل (Run)

main النقطة الرئيسية — java -cp "out;lib*" com.gym.cli.GymCliApp

مكتبات مجلد lib: sqlite-jdbc (تعريف JDBC لـ SQLite) و slf4j-api (سجلات المكتبة). و --enable-native-access تُسكِت تحذير الوصول للكود الأصلي في مشغل SQLite.

STARTUP FLOW

تدفق بدء التشغيل — GymCliApp.main

1

`initializeDatabase()`

ينشئ 13 جدولاً إذا لم تكن موجودة (CREATE TABLE IF NOT EXISTS)

2

`seedIfEmpty()`

يملأ بيانات تجريبية فقط إذا كان الجدول administrators فارغاً

3

طباعة مسار قاعدة البيانات

Using database file: <المسار الكامل> — شفافية: تعرف أي ملف gym.db يُستخدم

4

(Login Gate) شاشة الدخول

دخول يدوي أو Quick Login لثلاثة حسابات جاهزة

5

(Dashboard) لوحة القيادة

حلقة القوائم الرئيسية 1–10 + [V] + [L] خروج + [0] إنهاء

GymCliApp.java — نقطة الانطلاق ولوحة القيادة

src/com/gym/cli

ماذا يفعل هذا الملف؟

- main() ينشئ كائن التطبيق ويستدعي start().
- start(): تهيئة القاعدة + البذر + حلقة الدخول/الخروج.
- showLoginGate(): التحقق من اسم المستخدم وكلمة المرور عبر rules.authenticate (Rule 16).
- runMainDashboardLoop(): يعرض القوائم العشر ويوجّه الاختيار.
- printLiveKpiBanner(): أرقام حية (أعضاء، حضور اليوم، الإيراد...) أعلى اللوحة.
- حماية عامة: أي خطأ غير متوقع يُطبع بوضوح ويعود للقائمة بدلاً من إنهاء التطبيق.

لوحة القيادة الرئيسية

- [1] الأعضاء والملفات الطبية والتقدم
- [2] المدربون والمواعيد والاشتراكات
- [3] العضويات والخطط (بيع/تجميد/تجديد)
- [4] جدول الحصص والحجز
- [5] برامج التمرين ومحاكي HUD
- [6] الحضور (دخول/خروج + تقرير)
- [7] المدفوعات وسجل الإلغاء/الاستبدال
- [8] المعدات وخريطة الصالة
- [9] حسابات الموظفين [ADMIN فقط]
- [10] النسخ الاحتياطي وأدوات القاعدة
- [V] حزمة التحقق وقواعد العمل (1-19)

CliInput · CliSession · CliTheme

ثلاثي الأدوات المساعدة في الحزمة cli

CliInput.java

قراءة آمنة من لوحة المفاتيح

- readInt / readDouble / readBigDecimal — ترفض غير الأرقام وتعيد المحاولة.
- readIsoDate / readIsoTime — تفرض صيغة yyyy-MM-dd و HH:mm.
- readRequiredString — يرفض الفراغ. readYesNo — يقبل y/n فقط.
- كل حلقة تقرأ تنتهي بأمان عند نهاية الإدخال (EOF).

CliSession.java

حالة الجلسة الحالية

- currentUser: من المسجل دخوله الآن (دوره ورقمه).
- أعلام التنقل: logoutRequested / exitRequested / returnHome.
- shouldStopSubMenu(): سبب موحد لإنهاء أي قائمة فرعية.
- userLabel(): نص جاهز للعرض في ترويسة كل شاشة.

CliTheme.java

الطباعة المنسقة

- printHeader / printFooterNav — ترويسة كل شاشة بشكل موحد.
- printTable — جداول نصية بأعمدة مرتبة وقص آمن للنصوص الطويلة.
- printSuccess / printValidationError / printDatabaseError — رسائل ملونة دلالتها واضحة.
- safeText — يمنع null من أن يظهر كنص في الجداول.

src/com/gym/cli

GymRulesHelper.java — العقل المدبر لقواعد العمل

صنف واحد يجمع العمليات التي تحتاج أكثر من جدول أو أكثر من قاعدة — القوائم تستدعيه بدلاً من تكرار المنطق.

الحسابات والصلاحيات

- authenticate: دخول + رفض الحسابات غير النشطة (Rule 16)
- requireAdmin: حماية إجراءات الـ ADMIN (Rule 16)

العضويات

- freeze/unfreeze: نقل endDate بعدد أيام التجميد (Rule 7)
- renew: تجديد ويمنع CANCELLED (Rule 7)
- effectiveMembershipStatus: عرض EXPIRED بدون تعديل القاعدة

المدفوعات والرصيد

- computeOutstandingBalance: كل ما هو مستحق (Rule 6)
- voidAndReplacePayment: إلغاء + بديل بتحقق مسبق (Rule 19)

الأعضاء والحذف

- softDeleteCustomer: INACTIVE بدلاً من الحذف (Rule 4)
- hardDeleteCustomer/Trainer: يرفض إذا وُجدت مراجع (Rule 4)
- suspendCustomer: تجميد كل الخطط النشطة (Rule 14)

الحجز والجلسات

- bookOpenSlot: حجز مباشر ضمن السعة (Rules 1/13/17)
- requestOutsideSlots: خارج المواعيد يبدأ REQUESTED (Rule 1)
- confirm/complete/markNoShow: آلة حالات صارمة (Rule 18)
- cancelSession: سبب إلزامي + قاعدة ساعتين (Rule 10)

الحضور والملف الطبي

- checkIn: عضو ACTIVE + خطة سارية + صف واحد/يوم (Rule 5)
- getMedicalProfileForTrainer: للمدرّب المكلف فقط (Rule 15)
- exportSnapshot / loadSnapshot: نسخ .ser كامل

02

قواعد العمل التسعة عشر

BUSINESS RULES 1-19

قواعد العمل 1–10

رقم	القاعدة
Rule 1	الحجز داخل موعد منشور يبدأ CONFIRMED مباشرة، وخارجه يبدأ REQUESTED فينتظر التأكيد.
Rule 2	تسجيل حضور المدرّب للجلسات الفائتة: حالة NO_SHOW تُحصى في إحصاءات كل مدرّب.
Rule 3	ملف طبي واحد لكل عضو، يُنشأ عند الطلب فقط — التكرار مرفوض (عمود UNIQUE + تحقق).
Rule 4	الحذف الناعم يضع INACTIVE؛ والحذف النهائي ADMIN فقط ويُرفض وجود أي مراجع للعضو.
Rule 5	الدخول للحضور: عضو ACTIVE + خطة سارية + صف واحد فقط لليوم الواحد (إعادة الاستخدام).
Rule 6	الرصيد المستحق = خطط + اشتراكات مدرّب + جلسات مدفوعة، ناقص المدفوعات غير الملغاة.
Rule 7	التجميد للنشطة فقط، وإلغاء التجميد ينقل endDate بعدد الأيام، وتجديد الملغاة ممنوع.
Rule 8	الموعد المتاح: وقت البداية قبل النهاية + حد أقصى للعملاء أكبر من صفر.
Rule 9	كل التواريخ بصيغة ISO yyyy-MM-dd والأوقات HH:mm — تتحقق في كل DAO قبل الحفظ.
Rule 10	الإلغاء يتطلب سبباً، والإلغاء قبل ساعتين مجاني وإلا يُسجل كإلغاء متأخر مع السبب.

قواعد العمل 11–19

رقم	القاعدة
Rule 11	اشتراك المدرّب يجب أن يطابق المدرّب المكلف للعضو، والجلسة المشمولة بالاشتراك سعرها صفر (لا ازدواج فواتير).
Rule 12	كل دفعة مرتبطة بهدف واحد بالضبط (خطة أو اشتراك أو جلسة) + تطابق صاحب الدفعة + رقم إيصال فريد.
Rule 13	حالة الموعد تُشتق تلقائياً: CLOSED إن أُغلق، FULL عند اكتمال المحجوزين المؤكدين.
Rule 14	إيقاف العضو (SUSPENDED) يجمّد كل خططه النشطة تلقائياً.
Rule 15	قراءة الملف الطبي للمدرّب المكلف بالعضو فقط — أي مدرّب آخر يُرفض.
Rule 16	الصلاحيات: حساب غير نشط لا يدخل، وإجراءات حساسة تتطلب دور ADMIN.
Rule 17	سعة الموعد: عدد الجلسات CONFIRMED فيه يجب ألا يتجاوز maxCustomers.
Rule 18	آلة حالات الجلسة: REQUESTED ثم CONFIRMED ثم COMPLETED أو NO_SHOW أو CANCELLED — لا قفزات.
Rule 19	المدفوعات لا تُحذف أبداً: تُلغى (void) بسبب وتاريخ، مع إمكانية إنشاء دفعة بديلة برقم إيصال جديد.

[V] حزمة التحقق — ValidationSuiteRunner.java

ماذا تفعل؟

- تشغل 22 سيناريو تحقق تلقائياً تغطي الكيانات وقواعد العمل 1-19 ومعالجة الأخطاء.
- تعمل على قاعدة معزولة gym_sandbox.db: تُنشأ قبل التشغيل وتُحذف بعده — gym.db لا تُلمس.
- تطبع PASS/FAIL لكل سيناريو ثم لوحة نتيجة: SCOREBOARD n / 22.
- وضع ثانٍ: اختيار فئة واحدة (1-11) لعرض قاعدة محددة أمام الحضور.
- كل سيناريو معزول في try/catch: فشل واحد لا يوقف البقية (runCheck).
- إن نقصت بيانات البذر تطبع Seed data missing بدل الانهيار.

أمثلة سيناريوهات

#04 رفض اسم مستخدم مكرر (Rule 16)

#08 امتلاء الموعد ورفض الحجز الزائد (Rule 17)

#11 منع قفزة REQUESTED إلى COMPLETED (Rule 18)

#15 إعادة استخدام صف الحضور في نفس اليوم (Rule 5)

#20 الدفع بلا هدف مرفوض + Void يرفع الرصيد (Rules 12/19/6)

#22 نسخ .ser وتحميله والتقاط خطأ ملف مفقود

ملاحظة: النتيجة الحالية بعد الإصلاحات: 22 / 22 PASS.

DATA ACCESS LAYER

طبقة DAO — نمط واحد يتكرر 13 مرة

تشريح أي DAO (مثال CustomerDao)

`insert(obj)`

يُتحقق ثم INSERT + جلب المفتاح التلقائي

`update(obj)`

يُتحقق ثم UPDATE WHERE id=? + فحص عدد الصفوف

`delete(id)`

DELETE + فحص عدد الصفوف (بعضها يفحص المراجع أولاً)

`findById / findAll`

قراءة صف أو الكل عبر mapRow

`validate(obj)`

شرطات if بسيطة: فراغ، صيغة تاريخ/بريد، المراجع موجودة؟

`mapRow(rs)`

يحوّل صف ResultSet إلى كائن Model

ثلاث عادات ثابتة في كل DAO (اسألوا عنها!)،

`PreparedStatement + ? parameters`

لا دمج نصي لقيم المستخدم في SQL — حماية من SQL Injection وإدارة آمنة لعلامات الاقتباس (مثل اسم O'Brien).

`try-with-resources`

الاتصال والـ Statement والـ ResultSet يُغلقون تلقائياً — لا تسريب موارد حتى عند الخطأ.

`RETURN_GENERATED_KEYS`

بعد INSERT نقرأ المفتاح التلقائي rs.getInt(1) بدل max(id)+1 الهش.

نتيجة الفحص: كل عبارات SQL في المشروع بهذا النمط — لا استثناء واحد.

DAO PER TABLE

خريطة الـ DAOs الـ 13

DAO	الجدول	أبرز التحقق داخلها
CustomerDao	customers	تحقق كامل: اسم، بريد، هاتف 7 أرقام، تواريخ، وجود المدرّب المكلف
TrainerDao	trainers	تحقق: بريد/هاتف، خبرة غير سالفة، أجر غير سالف
MembershipDao	memberships	Rule 7: منع إعادة تنشيط الملغاة + تاريخ النهاية بعد البداية
TrainerAvailabilityDao	trainer_availability	Rule 8: بداية < نهاية + سعة > 0
TrainerSubscriptionDao	trainer_subscriptions	Rule 11: مطابقة مدرّب الاشتراك مع المكلَّف
SessionDao	sessions	Rules 9/11/18: صيغ + سعر صفر مع الاشتراك + آلة حالات
AttendanceDao	attendance	Rule 5: عضو ACTIVE + خطة سارية + صف واحد/يوم (حتى عند التعديل)
MedicalProfileDao	medical_profiles	Rule 3: ملف واحد لكل عضو (UNIQUE + تحقق مزدوج)
ProgressRecordDao	progress_records	مدى صحيح: وزن > 0 و دهون 0-100 و هدف > 0
PaymentDao	payments	Rule 12: هدف واحد بالضبط + تطابق المالك + إيصال فريد
WorkoutProgramDao	workout_programs	نسخة فريدة لكل عضو (excludeId عند التعديل)
EquipmentDao	equipment	صيانة بعد الشراء + حالات وأوضاع صحيحة
AdministratorDao	administrators	اسم مستخدم فريد (UNIQUE) + دور صحيح + حساب نشط

src/com/gym/util

DatabaseConnection.java — القلب التقني للقاعدة

المسؤوليات

- DEFAULT_DB_URL = jdbc:sqlite:gym.db — ملف واحد بجانب المشروع.
- initializeDatabase(): ينفذ 13 عبارة CREATE TABLE IF NOT EXISTS.
- getConnection(): اتصال جديد لكل عملية (نمط بسيط يناسب CLI أحادي المستخدم).
- busy_timeout = 5000: ينتظر حتى 5 ثوانٍ بدل الفشل الفوري لو الملف مقفول من عملية أخرى.
- setDbUrl / resetDbUrl: تُستخدم من حزمة التحقق للتحول للـ sandbox ثم العودة.

حقائق عن المخطط (Schema)

- كل جدول: id INTEGER PRIMARY KEY AUTOINCREMENT.
- قيود فريدة على مستوى القاعدة: username (موظفون)، receipt_no (مدفوعات)، customer_id (ملف طبي).
- التواريخ TEXT بصيغة ISO — لذلك المقارنة النصية (end_date >= ?) صحيحة.
- الأعلام true/false كـ INTEGER (0/1)، والأموال NUMERIC (تُقرأ BigDecimal).
- لا توجد FOREIGN KEY في SQLite هنا — البديل: فحوص وجود صريحة في الـ DAOs قبل الكتابة وقبل الحذف (Rule 4).

SEED DATA

SampleDataSeeder.java — بذور البيانات

ملف واحد يزرع بيانات تجريبية متسقة تُظهر كل الحالات: نشط، موقوف، غير نشط، منتهية، مجمدة، ملغاة، غياب (NO_SHOW)، دفعة ملغاة...

4 **حسابات موظفين** أحدها غير نشط لإظهار Rule 16	**3** **مدربون** تخصصات وأجور مختلفة	**5** **أعضاء** ACTIVE / SUSPENDED / INACTIVE	**5** **عضويات** ACTIVE / منتهية / CANCELLED / FROZEN
3 **مواعيد متاحة** بسعات 1-3 لعرض FULL	**4** **جلسات** CONFIRMED / REQUESTED / COMPLETED / NO_SHOW	**6** **معدات** بما فيها واحدة IN_MAINTENANCE	**4** **مدفوعات** منها زوج void + بديل (Rule 19)

دالتان فقط: seedIfEmpty() عند بدء التشغيل (إذا كانت القاعدة فارغة)، و resetAndSeed() من قائمة الأدوات [10] أو من حزمة التحقق بعد مسح الجداول.

com.gym.model

طبقة الموديلات — 14 كياناً و 11 نوعاً

النمط الموحد لكل كيان

- حقول خاصة private + مُنشئ فارغ + مُنشئ كامل (يسهل البذر والاختبار).
- getters/setters عادية — لا منطق مخفي داخل الموديل.
- equals/hashCode على المعرّف id فقط — تتسق مع مفتاح الجدول.
- implements Serializable + serialVersionUID = 1L لدعم النسخ .ser.
- الأموال BigDecimal (دقة مالية) والتواريخ String بصيغة ISO.
- الحقول الاختيارية Integer (مثل trainerId) — null = غير محدد.

الأنواع التعدادية (Enums) = حالات صارمة

CustomerStatus	ACTIVE · INACTIVE · SUSPENDED
MembershipStatus	ACTIVE · FROZEN · EXPIRED · CANCELLED
SessionStatus	REQUESTED · CONFIRMED · COMPLETED · NO_SHOW · CANCELLED
AvailabilityStatus	OPEN · FULL · CLOSED
SubscriptionStatus	ACTIVE · EXPIRED · CANCELLED
StaffRole	ADMIN · MANAGER · STAFF
PlanType	MONTHLY · QUARTERLY · YEARLY
PaymentMethod	CASH · CARD · TRANSFER
EquipmentStatus / Condition	AVAILABLE/IN_MAINTENANCE/RETIRED · NEW/GOOD/FAIR/POOR

EXCEPTION HIERARCHY

نظام الأخطاء — صنفان فقط يغطيان كل شيء



ValidationException

خطأ في المدخلات أو قاعدة عمل: بريد خاطئ، سعة ممتلئة، مكرر...

DatabaseException

خطأ تقني: فشل SQL، اتصال، قراءة/كتابة ملف .ser

دورة حياة الخطأ

- تُرمى من الـ DAO (تحقق) أو من GymRulesHelper (قواعد) مع رسالة واضحة.
- كل قائمة تلتقطها في catch وتطبعها بلون مناسب عبر CliTheme — دون انهيار.
- شبكة أمان أخيرة في لوحة القيادة: catch (Exception) يطبع الخطأ ويعود للقائمة.
- رسالة خطأ مفيدة دائماً: ماذا حدث + المعرف المتسبب (مثل: No trainer found with id: 9).

BINARY .SER BACKUP

النسخ الاحتياطي — SerializationUtil + GymBackupSnapshot

كيف يعمل؟

- GymBackupSnapshot: كائن واحد يحمل 13 قائمة (كل كيانات القاعدة) + تاريخ واسم المنشئ.
- saveToFile: ObjectOutputStream يكتب الكائن كاملاً في ملف .ser — ينشئ مجلد backups تلقائياً.
- loadFromFile: ObjectInputStream يقرأ ويتحقق instanceof GymBackupSnapshot.
- exportSnapshotToFile في GymRulesHelper يقرأ الـ 13 جدولاً عبر findAll ثم ينسخ.
- كل الموديلات Serializable — لذلك لا NotSerializableException.

الاستخدام والحدود

- القائمة [10]: تصدير نسخة، تحميل وفحصها، ومحاكاة أخطاء (ملف مفقود / مسار خاطئ) لعرض معالجة DatabaseException.
- serialVersionUID ثابت يجعل التحميل مستقراً بين التشغيلات.
- حد معروف: تعديل حقول أي موديل مستقبلاً يجعل النسخ القديمة غير قابلة للتحميل (طبيعة Java Serialization).
- بديل تعليمي ممكن: تصدير CSV نصي — أسهل للفحص اليدوي (قرار موثق في الملخص).

MENU FILES MAP

القوائم الست — خريطة سريعة

القائمة	الملف	أبرز الشاشات
الأعضاء	[1] Members	MemberCliMenu.java
المدربون	[2] Coaches	TrainerCliMenu.java
العضويات والمدفوعات	[3] & [7]	MembershipBillingCliMenu.java
الجدول والتمارين	[4] & [5]	ScheduleWorkoutCliMenu.java
المرافق والموظفون والأدوات	[6] [8] [9] [10]	FacilityAdminCliMenu.java
حزمة التحقق	[V]	ValidationSuiteRunner.java

نمط موحد في كل قائمة:

- حلقة while مع shouldStopSubMenu + ترويسة + خيارات مرقمة + [0] رجوع / [H] الرئيسية / [L] خروج.
- كل عملية داخل try/catch للخطأين المعروفين + إظهار رسالة ودودة ثم pause.

MENU [1]

MemberCliMenu.java — شاشات الأعضاء

الشاشات

- قائمة الأعضاء + بحث بالاسم/الهاتف/الرقم.
- Member 360 Profile: بطاقة تجمع كل ما يخص العضو (خطة، ملف طبي، تقدم، رصيد Rule 6).
- مدير الملف الطبي: إنشاء عند الطلب (Rule 3) وتعديل وحذف وتجربة وصول المدرّب (Rule 15).
- سجلات التقدم + رسم بياني ASCII لوزن العضو.
- إيقاف/إعادة تنشيط (Rule 14 يجمّد الخطط تلقائياً).

الحمايات هنا (بعد الإصلاح)

- التسجيل يسمح بفراغ المتعمَّد لعرض رسائل التحقق الودودة من CustomerDao.validate.
- الحذف الناعم الآن مع سؤال تأكيد قبل التنفيذ.
- حذف سجل تقدم يتطلب: موجود + يخص هذا العضو فقط + تأكيد — لا نجاح زائف أبداً.
- الحذف النهائي عبر rules.hardDeleteCustomer: ADMIN فقط + رفض عند وجود أي مراجع (Rule 4).

MENU [2]

TrainerCliMenu.java — شاشات المدربين

الشاشات

- قائمة المدربين مع عداد NO_SHOW لكل مدرب (Rule 2).
- ملف المدرب + قائمة الأعضاء المكلفين به (أساس Rule 15).
- المواعيد المتاحة: نشر، فتح/إغلاق (Rule 13)، تعديل السعة، حذف آمن.
- اشتراكات المدرب: إنشاء (Rule 11: مطابقة المكلَّف)، تجديد +30 يوم، إلغاء.
- جدول المواعيد يعرض المحجوز x/السعة والحالة المشتقة (FULL).

الحمايات هنا

- حذف موعد يُرفض إن فيه جلسات CONFIRMED — تُلغى أولاً (إصلاح يمنع جلسات يتيمة).
- تجديد الاشتراك يتعامل بأمان مع تاريخ نهاية تالف مخزّن (try/catch حول LocalDate.parse).
- إلغاء اشتراك غير موجود يطبع رسالة واضحة بدل الصمت.
- التحقق في TrainerDao يمنع أجراً سالباً وخبرة سالبة وبريداً خاطئاً.

MENUS [3] & [7]

MembershipBillingCliMenu.java — العضويات والمدفوعات

شاشة العضويات [3]

- بيع خطة (MONTHLY/QUARTERLY/YEARLY) مع أسعار مقترحة ونهاية محسوبة تلقائياً.
- تجميد نشطة + إلغاء تجميد ينقل endDate (Rule 7).
- تجديد يرفض الملغاة، وإلغاء مباشر بحالة CANCELLED.
- الجدول يعرض الحالة الفعلية (Effective) المحسوبة ليوم اليوم — المنتهية تظهر EXPIRED دون تعديل القاعدة.

شاشة المدفوعات [7]

- تسجيل دفعة: هدف واحد بالضبط + تطابق المالك + إيصال فريد (Rule 12).
- إدخال يدوي بالروابط للمتقدمين — نفس التحقق يحميه.
- بحث برقم الإيصال + الرصيد المستحق التفصيلي (Rule 6).
- Void & Replace (Rule 19): تحقق من الإيصال البديل قبل إلغاء القديمة — لا نصف عملية أبداً.

ScheduleWorkoutCliMenu.java — الجدول والتمارين

شاشة الجدول [4]

- حجز موعد منشور = CONFIRMED فوراً بضوابط 1/5/13/17.
- طلب خارج المواعيد = REQUESTED ثم تأكيد (Rule 18).
- حجز استثنائي للـ ADMIN يتجاوز السعة (Rules 1/16).
- إلغاء بقاعدة الساعتين وسبب إلزامي (Rule 10).
- عند حجز باشتراك: السعر يُفرض صفراً (Rule 11).

شاشة التمارين [5]

- إنشاء برنامج تمرين (تمارين نصية بتمارين وتكرارات).
- نشر نسخة جديدة v+1 — النسخة فريدة لكل عضو (تحقق excludeId).
- عرض التمارين وإعادة إسناد المدرب المنشئ.
- Live Workout HUD: محاكي تفاعلي يمر على التمارين بنِسَب تقدم.
- حذف برنامج مع وجود و تأكيد — لا نجاح زائف.

MENUS [6] [8] [9] [10]

FacilityAdminCliMenu.java — المرافق والموظفون والأدوات

[6] الحضور

- دخول سريع بضوابط Rule 5
- خروج + تقرير يومي
- تصحيح وقت صف موجود (لا حذف — Rule 19)

[8] المعدات

- إضافة/تعديل حالة ووضع
- حذف بوجود وتأكيد
- خريطة صالة ASCII بأربع مناطق

[9] الموظفون

- إضافة حساب + أدوار (Rule 16)
- تنشيط/إيقاف + تعديل كلمة المرور
- منع تدمير حسابك بنفسك (إيقاف/حذف/تخفيض دور)

[10] الأدوات

- تصدير/تحميل نسخة .ser
- محاكاة أخطاء قاعدة البيانات
- تصفير القاعدة وإعادة البذر

03

الإصلاحات وأسئلة المناقشة

FIXES & EXPECTED QUESTIONS

ماذا أصلحنا ولماذا (1/2) — الأخطاء التي كانت تُنهي التطبيق

المكان	الخطأ القديم	الإصلاح
`GymRulesHelper`	تاريخ تالف في القاعدة كان يُسقط التطبيق كاملاً عند تجميد/تجديد/إلغاء	تحليل تواريخ محمي (parseStoredDate/Time) برسالة تحقق بدل انهيار DateTimeParseException
`GymCliApp`	أي RuntimeException كانت تُنهي main بكومة أخطاء	شبكة أمان catch (Exception) حول قوائم اللوحة — خطأ غير متوقع يُطبع ويعود للتطبيق
`CliInput`	ضغط Enter بلا قيمة افتراضية = حلقة لا نهائية من رسالة خطأ	حماية Enter الفارغ في readDouble/readBigDecimal/readIsoDate/readIsoTime قبل المحاولة
`ValidationSuiteRunner`	نقص بيانات البذر كان يعني NPE وإنهاء التطبيق وتوقف اللوحة	فحص null بعد كل findById في حزمة التحقق + عزل كل سيناريو (runCheck)
`TrainerCliMenu`	نفس عائلة انهيار التواريخ في شاشة المدربين	تجديد الاشتراك بـ try/catch حول LocalDate.parse

ماذا أصلحنا ولماذا (2/2) — التحقق وسلامة البيانات

المكان	الخطأ القديم	الإصلاح
كل القوائم	'Deleted #999' تُطبع حتى للمعرفات غير الموجودة	كل حذف يتحقق من الوجود أولاً (+ تأكيد) — ولن يطبع نجاحاً زائفاً بعد الآن
Member/Trainer	عضو كان يحذف سجلات غيره؛ حذف موعد يتيم الجلسات	حذف سجل التقدم يتطلب ملكية العضو نفسه؛ حذف موعد فيه CONFIRMED مرفوض
Rule 19 flow	إلغاء يُنفذ ثم يفشل البديل = نصف عملية؛ ورسالة نجاح كاذبة	Void & Replace يتحقق من إيصال البديل قبل الإلغاء + الإيصال إلزامي
AttendanceDao	تعديل تاريخ حضور كان يصنع صفين ليوم واحد	فحص Rule 5 حتى عند التعديل (sameDayRowExists مع excludeId)
GymRulesHelper	حجز الماضي والموقوف كان ممكناً	منع حجز موعد في الماضي وعضو غير ACTIVE + فحص حالات Rule 18 الصريح
13 DAO + DB	عمليات صامتة بلا أثر وأخطاء تقنية غامضة	فحص عدد الصفوف في update/delete لكل الـ DAOs + رسائل ودودة لقيود UNIQUE + busy_timeout + طباعة مسار القاعدة

أسئلة متوقعة وأجوبة قصيرة (1/2)

س: لماذا استخدمنا نمط DAO؟

ج: لفصل الوصول للبيانات عن الواجهة. تغيير قاعدة البيانات يمس الـ DAOs فقط، والقواعد تُختبر بمعزل عن SQL.

س: لماذا PreparedStatement وليس نص SQL مدموج؟

ج: حماية من SQL Injection، وتعامل آمن مع الاقتباسات والأنواع، وقابلية إعادة استخدام العبارة.

س: ما الفرق بين ValidationException و DatabaseException؟

ج: الأول: خطأ مستخدم/قاعدة عمل (يراجع المستخدم مدخلاته). الثاني: خطأ تقني (SQL/ملف) يحتاج انتباه المطور.

س: لماذا try-with-resources؟

ج: يغلق Connection/Statement/ResultSet تلقائياً حتى عند الاستثناء — يمنع تسريب الموارد دون كتل finally يدوية.

س: كيف تُطبق Rule 17 (سعة الموعد)؟

ج: SELECT COUNT للجلسات CONFIRMED في الموعد، إن وصل maxCustomers تُشتق حالة FULL ويُرفض الحجز.

س: كيف تعمل آلة حالات الجلسة (Rule 18)؟

ج: validateTransition في SessionDao يرفض أي انتقال غير قانوني: مثلاً REQUESTED لا تصبح COMPLETED إلا عبر CONFIRMED.

أسئلة متوقعة وأجوبة قصيرة (2/2)

س: كيف نمنع حذف بيانات مرتبطة؟

ج: حارس Rule 4 يعدّ المراجع في كل الجداول قبل الحذف — وجدت واحدة يُرفض الحذف ويقترح الحذف الناعم INACTIVE.

س: ما فائدة قاعدة sandbox في حزمة التحقق؟

ج: قاعدة مؤقتة gym_sandbox.db: تُنشأ وتُبذر وتُختبر ثم تُحذف — التحقق الشامل دون أي خطر على gym.db.

س: لماذا implements Serializable؟

ج: لحفظ نسخة كاملة (ser.) من كل الكيانات بكائن واحد (GymBackupSnapshot) عبر ObjectOutputStream/ObjectInputStream.

س: ماذا يحدث عندما تنتهي صلاحية خطة؟

ج: لا شيء يُعدل — effectiveMembershipStatus يحسب EXPIRED مقارنة endDate باليوم، فالعرض صادق والقاعدة نظيفة.

س: كيف يعمل التجميد (freeze)؟

ج: الحالة تصبح FROZEN مع عدد الأيام؛ وعند unfreeze ينتقل endDate للخلف بمقدار أيام التجميد فلا يخسر العضو يوماً.

س: لماذا التواريخ TEXT بصيغة ISO؟

ج: لأن yyyy-MM-dd يرتب نصياً كما يرتب زمنياً — فمقارنة end_date >= ? صحيحة بدون دوال تواريخ خاصة.

سيناريو العرض العملي — 4 دقائق

1

شغلوا run.bat — اعرضوا طباعة مسار gym.db ثم Quick Login كـ ADMIN

2

من [V] شغلوا الحزمة الكاملة (1) — SCOREBOARD 22/22 أمام الجميع

3

جولة سريعة: [1] عضو 360 + رصيد، [4] حجز موعد، [7] دفعة ثم Void

4

أنهِ بـ [0] ثم [0] — خروج نظيف بلا أي رسالة خطأ

ملخص كلمة أخيرة

مشروع متكامل بأربع طبقات واضحة: قوائم تدخل، RulesHelper يقرر، DAO يحفظ، SQLite تخزن — و19 قاعدة تحمي البيانات في كل اتجاه. كل ملف له دور واحد يسهل شرحه.

بالتوفيق في المناقشة!